Source: docs\COUPON_LOYALTY_AUDIT.md | Export: 11 November 2026

Coupon and Loyalty Points audit - 9 November 2026

Local implementation based on commit `c57ccec860145790792f314e6302e72332294c3c`. No production records, payment settings, credentials, rules or indexes changed. No real Razorpay payment was initiated. No commit, push or deployment performed.

Confirmed findings and fixes

Finding	Local result
Checkout requested `eligibleOnly: true`	Checkout now requests the complete public discovery result.
Discovery omitted exhausted offers and customers past an exact order sequence	Active public offers stay visible with a safe reason and disabled Apply.
Coupon title/description had no schema or Admin fields	Optional public title (100 characters) and full plain-text description (2,000 characters) round trip through the existing editor and save API. Legacy coupons remain valid without invented text.
Checkout had no Copy action and omitted copy status/error output	Every displayed coupon has Copy; checkout announces success or failure.
Final validation gave a generic rejection for dates, limits and minimum	Discovery and trusted checkout share specific safe rejection reasons; discount arithmetic and transaction boundaries are unchanged.

Product discovery remains an initially closed dropdown, showing the selected offer in full. Checkout keeps its existing expandable Available Coupons panel. Shop remains free of coupon lists. Both themes and existing Lora typography remain. Ineligible offers cannot be applied; eligible offers still require a trusted checkout review. A stale applied coupon is retained and progression blocked, with the server reason and Remove action available.

Publication means the existing `discoverable: true` setting. Private/missing publication, disabled, scheduled and expired codes remain excluded. Exhausted codes remain visible as Not Eligible. Only public terms/text and the customer's own safe eligibility result are returned, never customer identifiers, usage records, internal notes or payment evidence.

Server authority and loyalty audit

Area inspected	Existing behavior preserved and checked locally
Admin changes	Both admin claim and enabled access record; strict input schema; revision conflicts; idempotent saves; usage preserved.
Coupon calculation	Integer paise/BigInt, whole percentages, floor rounding, percentage cap, fixed discount clamped to merchandise, dates and minimum checked server-side. Shipping remains separate.
Customer eligibility	Verified token UID, exact prior Delivered-order requirement, per-customer/global usage and one accepted use for sequence coupons. Browser-supplied eligibility/amounts/UID are rejected.
Acceptance/concurrency	Final reads, order, coupon usage, stock and loyalty reservation commit in one transaction. Competing accepts cannot exceed limits; exact retries return the existing result.
Earning	Eligible Delivered transition credits once with stable ledger IDs. COD collection is independent; online orders require trusted captured payment and consumed reservation. No earning on failed payment.
Redemption	Server-owned rates, minimum and percentage cap; coupon compatibility setting; integer conversion and allocation. Quotes reserve nothing. Insufficient requests are reduced with a warning, and a changed quote requires reconfirmation before acceptance.
Balance/ledger	Private UID-based account/history, reserved/pending/spendable values, owned pagination, operation receipts and ledger reconciliation. Two orders cannot spend the same available balance.

Area inspected	Existing behavior preserved and checked locally
Cancellation/recovery	COD cancellation/rejection restores reserved points and pending earnings; accepted COD coupon usage remains consumed under existing policy. Uncaptured online failure retains the reservation for safe retry, then trusted expiry releases stock, coupon and points exactly once.
Returns/refunds	Allocated COD refunds restore/reverse points proportionally. Already-spent earnings can produce recorded debt; spendable availability stays zero. Provider-confirmed online refunds freeze redemption pending trusted allocation/review.
Screens/security	Existing Account/Admin balances and history, Apply/Remove Points, checkout warnings and order snapshots; no direct customer balance writes or admin balance-edit bypass.

No new loyalty lifecycle bug was reproduced in the covered cases. The lifecycle was retained; new regression assertions check failed-payment ledger conservation and the redemption block after an unallocated provider refund.

Scope limitations

- Customer allowlists and product/category coupon targeting have no existing schema, Admin controls or enforcement policy. The current supported customer restrictions are usage limits and exact Delivered-order sequences. These unsupported targeting options were not represented as implemented or added speculatively.
- There is no independent draft/published coupon status: `discoverable` controls public discovery, while enabled/date/usage/sequence rules control application.
- There is no manual balance editor, automatic historical points backfill, expiry, transfers or referrals. Existing history was not rewritten.
- A provider refund is insufficient to infer returned merchandise allocation. The existing online-refund points freeze needs trusted owner reconciliation; this audit does not add an automatic allocation or an unsafe unfreeze control.
- No rates, public descriptions or visibility switches were populated in Production. The owner supplies approved public copy through Admin after release.

Checks and evidence

- `npm.cmd test`: 194 passing, zero failed/skipped.
- Nineteen backend/rules suites: 41 passing tests including nested payment and cutover checks; zero failed/skipped. Coverage includes exported handlers, ownership, dual admin authorization, coupons, loyalty, delivery earning, COD, mocked online capture/recovery/refunds, inventory, settings and shopping.
- `npm.cmd run test:catalog-runtime`: packaged Vercel handler passed with and without require(ESM).
- `npm.cmd run test:payments-runtime`: packaged Web exports rejected signed-out requests and invalid configuration. The payment emulator also tested original webhook bytes, valid HMAC acceptance and tamper rejection through the artifact.
- Eight focused browser suites passed: checkout coupons, themes/offers, Admin coupons, loyalty, payment presentation, checkout design, checkout context recovery, and Wishlist/Bag. Coupon screens were checked in both themes at 320/390/768/1440px; existing loyalty checks extend to 1920px. Coverage includes keyboard opening, Copy/announcements, disabled Apply/reasons, changing subtotal and Delivered history, invalid applied-code retention, empty/error/retry, Apply/Remove Points, account changes and late accepted-response recovery.
- Representative 390px coupon and product screenshots, a 1440px checkout screenshot, and the 320px Admin editor screenshot were visually inspected. Full descriptions wrap, white surfaces remain readable and actions remain reachable around the existing fixed controls.
- Final `npm.cmd run lint`, `npm.cmd run build` and `git diff --check` passed; the build reports its existing large-chunk warning.

Initial browser runs caught test assumptions that no longer matched the requested behavior: guest Apply was previously absent rather than disabled; eligibility text now includes its explicit label; the request observer now includes earlier Bag discovery. Assertions were corrected to verify disabled actions, the complete label/reason and no new reads from a closed checkout selector. None were skipped or disabled. The final eight suites passed after these corrections.

Local logs and screenshots are under `.cache/coupon-loyalty-audit-*.log`, `.cache/checkout-coupons`, `.cache/theme-offers` and `.cache/coupons-design`. All browser fixtures use loopback services; provider requests are mocked or intercepted. Emulator/rules success does not establish Production index/rule readiness, deployed configuration, or real Razorpay TEST/LIVE success.

Changed files

- `src/orders/model.ts`: backward-compatible public metadata and shared reasons.
- `src/offers/model.ts`, `AvailableCoupons.tsx`, `offers.css`: full discovery, eligibility UI, Copy feedback and responsive description surfaces.
- `src/admin/coupons/model.ts`, `CouponEditor.tsx`, `coupons.css`: public text editing, validation and narrow textarea styles.

- `src/admin/coupons/model.test.ts`: pure model/validation regressions.
- `scripts/coupons-emulator.test.ts`, `theme-offers-emulator.test.ts`, `payments-emulator.test.ts`: real-service metadata, projection, ownership, discovery, payment reservation and ledger regressions.
- `scripts/theme-offers-fixture-control.ts`: loopback-only test scenarios.
- `scripts/checkout-coupons-browser.test.py`, `theme-offers-browser.test.py`, `coupons-browser.test.py`: customer/Admin interaction regressions.
- `docs/COUPONS.md`, `docs/LOYALTY.md`, this report: behavior and evidence.

Release boundary

These changes remain local until the owner gives a separate release instruction. No new Firestore rule/index publication or environment variable is required by this diff. Production settings and hosted acceptance have not been re-audited. After an approved release, check public/private offer discovery, copied codes, invalid applied-code feedback, Apply/Remove Points and customer/Admin history through existing authorized sessions. Any real payment test requires separate authorization; local mocks must not be reported as payment acceptance evidence.